



Introduction to NLP

AIAA 4051 Lecture 2

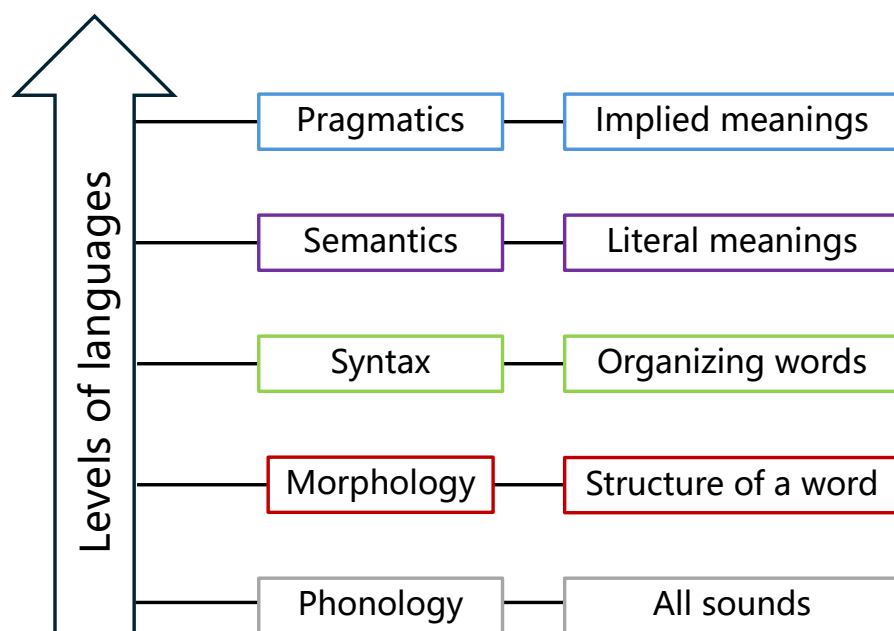
Instructor: Sihong Xie

Email: sihongxie@hkust-gz.edu.cn

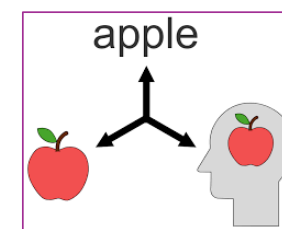
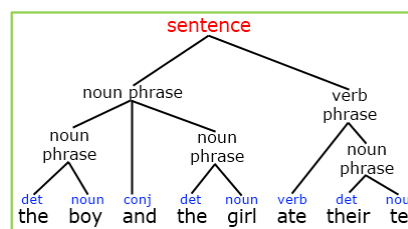
AI Thrust, HKUST(GZ)

Elements in NLP

- There is a hierarchy of linguistic elements.



Use of metaphor, irony, sarcasm, humor, jokes, silence



unhappiness: un-happi-ness housewife: house-wife

e	ə	ɜ:	ɔ:
bed	teacher	bird	door

Let's start with the basic elements

- Words*

- Word tokenization**: Isolating a word from a sentence

```
# Tokenizing sentences into words

sentence = "Hello world, this is a simple example of tokenization."

# Method 1: Using split()
words = sentence.split()    # use spaces to split the sentence
print("Method 1 - split():")
print(words)

# Method 2: Using regex
import re
# \b denotes word boundaries (e.g., space, beginning of a word)
# \w+ matches one or more word characters
words = re.findall(r'\b\w+\b', sentence)
print("\nMethod 2 - regex:")
print(words)

# Method 3: Using NLTK
from nltk.tokenize import word_tokenize
# can keep the punctuations as separate tokens
words = word_tokenize(sentence)
print("\nMethod 3 - NLTK:")
print(words)
```

Output:

Method 1 - split(): ['Hello', 'world,', 'this', 'is', 'a', 'simple', 'example', 'of', 'tokenization.']

Method 2 - regex: ['Hello', 'world', 'this', 'is', 'a', 'simple', 'example', 'of', 'tokenization']

Method 3 - NLTK: ['Hello', 'world', ',', 'this', 'is', 'a', 'simple', 'example', 'of', 'tokenization', '.']

Attaching a comma (,) to "world" is obviously undesirable.

keeping the punctuations can be important for some tasks.

* We assume that the words are those in English; no study of characters (not interesting) and sounds (part of speech processing).

** tokenization means separating something into smaller pieces, not necessarily words.

Let's start with the basic elements

- Words*
 - Destruction of a word

```
def split_on_delimiters(token):
    # split on hyphens and underscores
    parts = re.split(r'[_-]', token)
    return [p for p in parts if p]

def split_camel_case(token):
    # insert space before capital letters
    s = re.sub(r'([a-z0-9])([A-Z])', r'\1 \2', token)
    return s.split()

def split_alphanum(token):
    # split into alphabetic, numeric, and punctuation parts
    return re.findall(r'[A-Za-z]+\d+|[\^\w\s]', token)

def subword_tokenize(token):
    # keep punctuation as-is
    if re.match(r'^\W+$', token): # only non-word characters
        return [token]
    parts = []
    for p in split_on_delimiters(token): # handling snake_case_example
        for q in split_camel_case(p): # handling camelCaseWord & userID42
            parts.extend(split_alphanum(q)) # handling userID42
    return parts
```

```
# Use existing `words` and `sentence` variables from the notebook
subwords = {w: subword_tokenize(w) for w in words}

# Examples with common composite words
composite_examples = [
    'state-of-the-art', 'camelCaseWord', 'snake_case_example',
    'COVID19', 'anti-inflammatory', 'mother-in-law', 'e-mail', 'userID42'
]

# Print results
print("From existing 'words':")
for w, s in subwords.items():
    print(f"{w!r} -> {s}")

print("\nComposite examples:")
for w in composite_examples:
    print(f"{w!r} -> {subword_tokenize(w)}")
```

Composite examples:

'state-of-the-art' -> ['state', 'of', 'the', 'art']

'camelCaseWord' -> ['camel', 'Case', 'Word']

'snake_case_example' -> ['snake', 'case', 'example']

'COVID19' -> ['COVID', '19']

'anti-inflammatory' -> ['anti', 'inflammatory']

'mother-in-law' -> ['mother', 'in', 'law']

'e-mail' -> ['e', 'mail'] 'userID42' -> ['user', 'ID', '42']

n-gram language models

- Language models

- Define the probability of a sentence $P(w_1, \dots, w_n)$

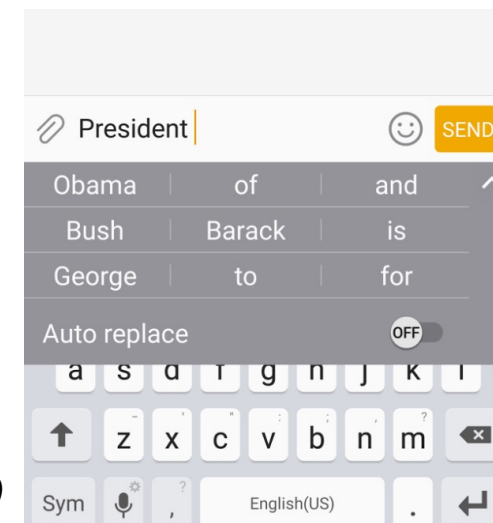
- Spell-checking: $P(\text{"the book"}) \gg P(\text{"book the"})$
 - Predict the next word: $P(\text{"Obama"}|\text{"President"}) \gg P(\text{"book"}|\text{"President"})$

- The joint distribution requires *exponentially* many data and memory to estimate well, and many ways to break this distribution:

- Uni-gram: $P(w_1)P(w_2) \dots P(w_n)$

- Bi-gram: $P(w_1)P(w_2|w_1) \dots P(w_n|w_{n-1})$

- Where do these probabilities come from?



| n-gram language models

- The model $\theta = \{P(w) : w \in V\}$
 - Uni-gram language model: words are selected independent of other words.
 - The likelihood of the corpus is

$$P(w_1, \dots, w_n) \propto P(w_1) \times \dots \times P(w_n) \propto \prod_{w \in V} P(w)^{c(w)} = L(w_1, \dots, w_n | \theta)$$

- The probability of observing a word w is
- Then, we can count the number of times any word $w \in V$ appears.
- Denote the count as $c(w)$.
- We know the MLE of $P(w)$ is just $c(w)$ divided by the total number of tokens.

| n-gram language models

- Bi-gram: conditional probability of a word given the previous word

$$P(w|v) = \frac{\text{Count of } (vw)}{\text{Count of } v}$$

- Tri-gram: conditional probability of a word given the previous two words $P(w|uv) = \frac{\text{Count of } w}{\text{Count of } uvw}$

- Example

- Estimate $P(\text{Sam}|\text{am})$ and $P(\text{Sam}|\text{I am})$

*The corpus = {
D1=[I am Sam]
D2=[Sam I am]
D3=[I do not like green eggs and ham]
}*

- Question: is $P(\text{eggs}|\text{brown})=0$?

| n-gram language models: smoothing

- Missing words and phrases

- Laplacian smoothing for unigram $P_{\text{Laplacian}}(w) = \frac{(\text{Count of } w) + 1}{(\text{Count of the total tokens}) + V}$
- Verify that the smoothed estimation is a distribution over the vocabulary.
- Laplacian smoothing for bi-gram $P_{\text{Laplacian}}(w_2|w_1) = \frac{(\text{Count of } w_1 w_2) + 1}{(\text{Count of } w_1) + V}$
- Example:

*The corpus = {
D1=[I am Sam]
D2=[Sam I am]
D3=[I do not like green eggs and ham]
}*

$$P_{\text{Laplacian}}(w = \text{"am"}) = \frac{2 + 1}{14 + 10}$$

$$P_{\text{Laplacian}}(w = \text{"am"} | \text{"I"}) = \frac{2 + 1}{3 + 10}$$

Symbolic vs. semantics

- Now we have tokens as symbols, but what are symbols?

- Given a list of binary sequences

- 01001000 01100101 01101100 01101100 01101111

symbols

- A human or computer does not know what it is (meaning/semantics).

- If it is decoded using ASCII, it is "Hello";
- If it is decoded using RGB, it represents a color;
- If it is decoded as a wave signal, it represents a soundwave of length 0.01 second.

semantics

- Chinese room thought-experiment^[1]

- Someone, say John, who does not know Chinese, uses a Chinese-English rule book in a room to answer questions in Chinese (what foods best pair with beef?)
- People outside the room will think that the person must know the meaning of the questions.
- But John just operates symbols rather than semantics.

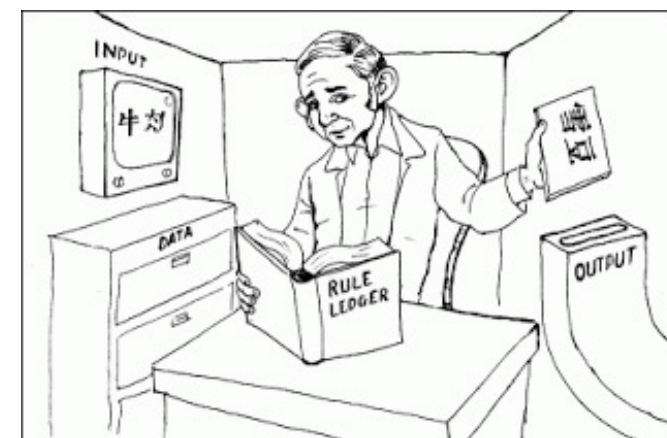


Image source: <https://theness.com/neurologicablog/ai-and-the-chinese-room-argument/>

[1] https://en.wikipedia.org/wiki/Chinese_room

| Symbolic vs. semantics

- Why computers (including LLMs) are inherently symbolic?
 - Computers use low-level elementary physical signals (voltage) to compose and represent high-level concepts.
 - No meaning associated with the voltage or the compositions.
 - Humans are different: form semantics in brain via interaction with the real-world with many senses (touch, hearing, temperature, pain, etc.).
 - Philosophy question: humans also use physical signals on the biological level, and how “semantics” is defined and perceived?

Levels	Semantics (Human perspective)	Symbolic (Computer perspective)
Physical	Circuit open and close	High/Low Voltage (3.3V/0V)
Logical	0 means off, 1 means on	0/1
Symbolic	“A” means Apple or Alphabet	01000001
Semantics	“cat”: animal that’s fluffy	01100011 01100001 01110100

Image source: <https://medium.com/data-science/a-chinese-speakers-take-on-the-chinese-room-88a0558b2cc8>

Word semantics*

- Use a dictionary
 - an entry is a lemma;
 - can have multiple senses;
 - each sense can have examples of using the lemma.
- Use WordNet
 - Basic unit: word senses to represent word **semantics**.
 - Synsets: synset("good-1") = {"excellent", "great", ...}.
 - Hypernyms: broader meanings.
 - Hyponyms: more specific meanings.

Advantages: represent human knowledge; sort of comprehensive; encode some relations.

Disadvantages: meanings become outdated easily; not so comprehensive and cannot include new words; **mostly symbolic**.

Image source: <https://phdtopic.com/wordnet-projects/>

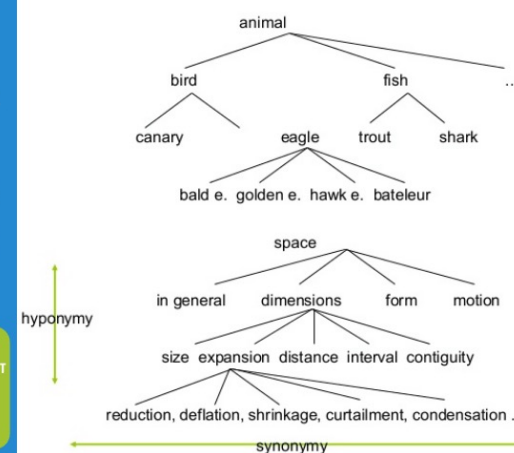


Famous English dictionaries:

- Merriam Webster
- Cambridge Dictionary

Famous Chinese dictionaries:

- 说文解字
- 新华字典



* We will discuss syntax in later lectures.

** Sentences, sub-words, paragraphs also have semantics.

One-hot word vector

- From here on, we use token from here on to refer to basic elements of sentences like words/sub-words. Vocabulary=the set of all tokens.
- Let each word/sub-word occupy one column and one row of a matrix.
 - Each row represents a word with one and only one 1 (others are zeros).
- Examples

Vocabulary={'rock', 'classic', 'pop'}

'rock' → [1, 0, 0]

'classic' → [0, 1, 0]

'pop' → [0, 0, 1]

Vocabulary={'cat', 'dog', 'bird', 'fish'}

'cat' → [1, 0, 0, 0]

'dog' → [0, 1, 0, 0]

'bird' → [0, 0, 1, 0]

'fish' → [0, 0, 0, 1]

Vocabulary={'cat', 'dog', 'bird', 'fish', 'bear', 'tiger'}

'cat' → [1, 0, 0, 0, 0, 0]

'dog' → [0, 1, 0, 0, 0, 0]

'bird' → [0, 0, 1, 0, 0, 0]

'fish' → [0, 0, 0, 1, 0, 0]

'bear' → [0, 0, 0, 0, 1, 0]

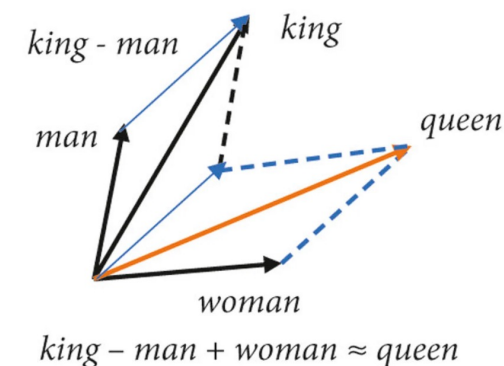
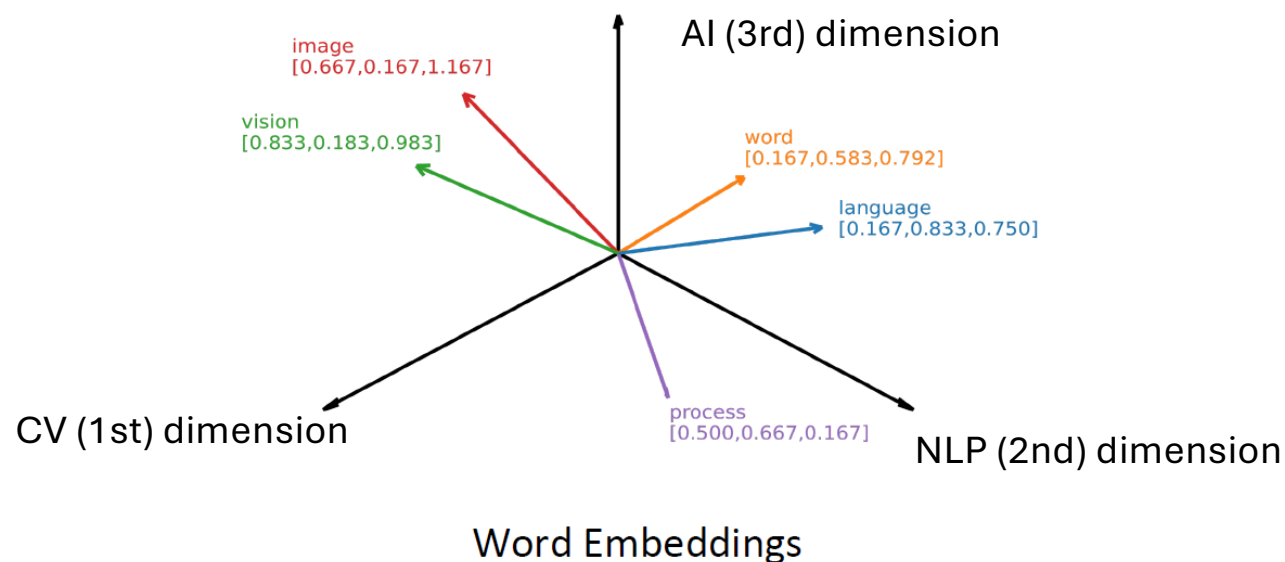
'tiger' → [0, 0, 0, 0, 0, 1]

Advantages: unique ID of a word, so that Beijing != 2 x Shanghai

Disadvantages: semantically related words 'cat' and 'tiger' have 0 similarity; the location of 1 cannot represent multiple aspects of a word ('cat' is cute, has hair, is a pet, is of the same family as 'tiger', etc.)

Distributional word vector

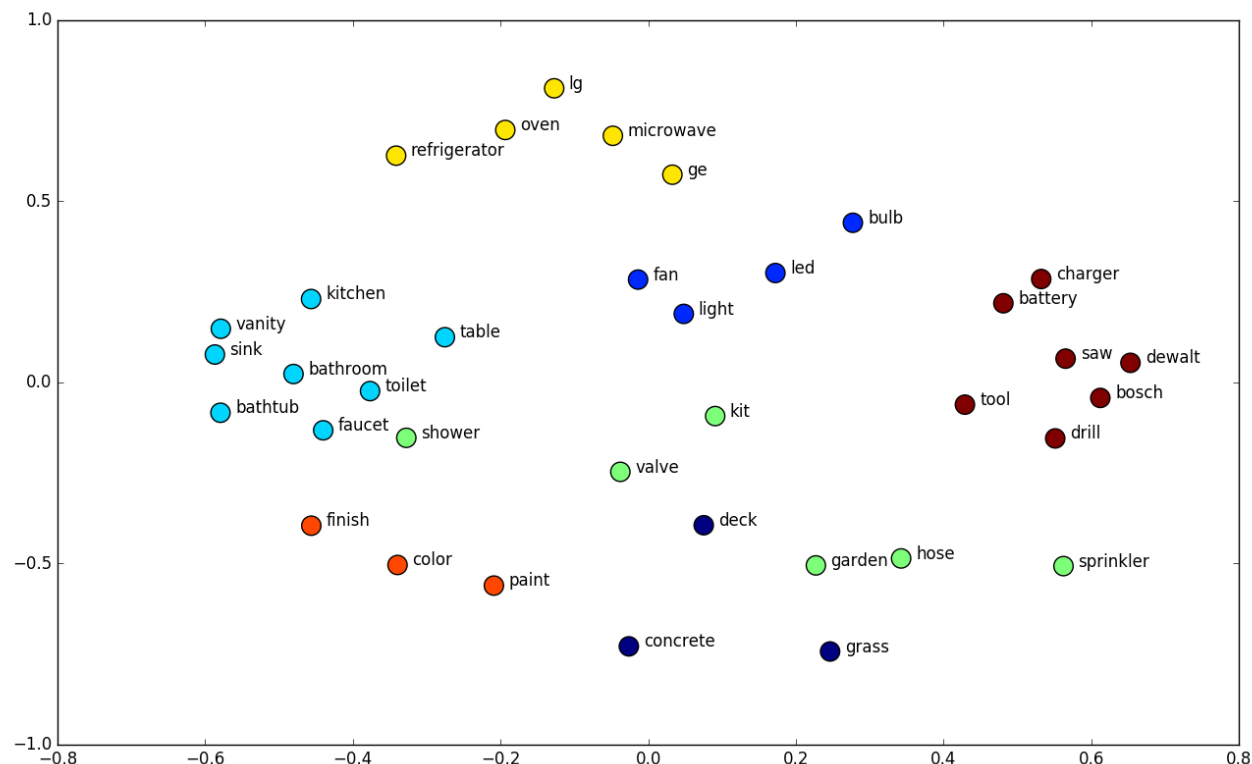
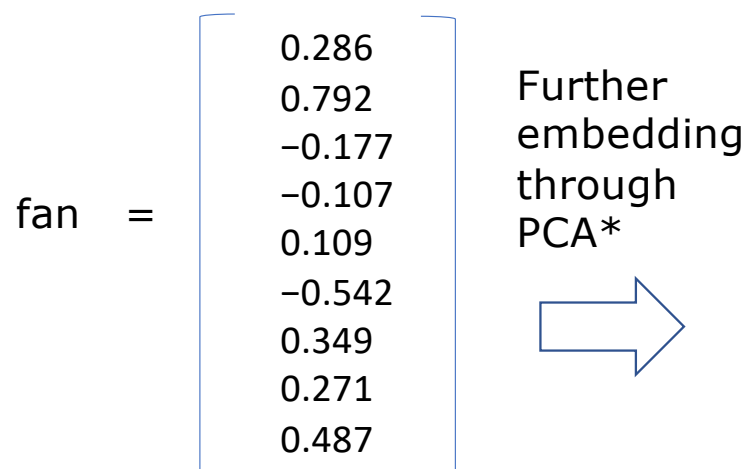
- A word vector (or embedding) of low (50 ~ 300) dimensions.
 - Each dimension represents some meaning: a number in one dimension of the vector means some kind of association strength.



Allow for linear algebra operations to represent semantic transformation: critical for computer/GPUs.

Image source: <https://medium.com/@prasanNH/word-embeddings-bridging-semantics-and-distributional-similarity-ff573c1e7f9d>

Visualization of word vectors



* PCA: Principle Component Analysis to reduce a high dimensional vector to a lower-dimensional (e.g., 2) space by maximally preserve data variance.

Image source <https://www.shanelynn.ie/get-busy-with-word-embeddings-introduction/>

Learn word vectors from text data

- A very successful principle in NLP

"You shall know a word by the company it keeps" (J. R. Firth 1957: 11)

- e.g., nations' names appears in the context of presidents' or capital cities' names.
- Context of a word w :
 - other words that appear near w reflecting what w means.
 - "near" can be defined by a window of 5 words, a sentence, or a document.

The corpus = {
D1=[I am Sam]
D2=[Sam I am]
D3=[I do not like green eggs and ham]
}

a small window:

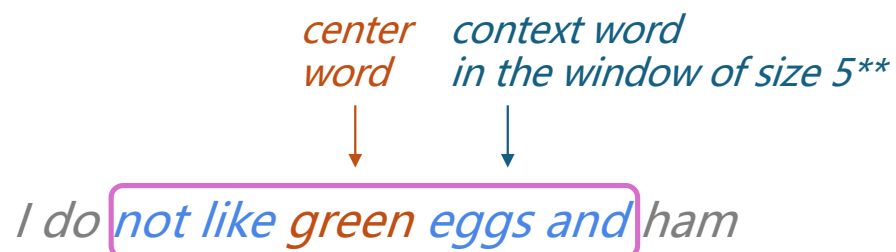
$\text{context}("I") = \{"am", "do", "Sam"\}$

a larger window:

$\text{context}("I") = \{"am", "do", "Sam", "not"\}$

Word2Vec: intuition of skipgram*

- Find **positive** pairs of (center, context) words in a sliding window:



- Want the vector of *green* to be predictive of *eggs* with high probability

- $P(\textit{like} \mid \textit{green}) = P(\textit{eggs} \mid \textit{green})$ Closest context words have the highest probabilities
- $> P(\textit{not} \mid \textit{green}) = P(\textit{and} \mid \textit{green})$ Farther-away context words have lower probabilities
- $> P(\textit{I} \mid \textit{green}), P(\textit{do} \mid \textit{green}), P(\textit{ham} \mid \textit{green})$ Outside context words have lowest probabilities

Negative pairs for this sliding window

* predicting central words using context words is called “CBOW”

** fun facts: modern GPTs use super large window sizes like 128k

Word2Vec: modeling

- Formally, let each word w have two parameter vectors
 - $\mathbf{v}_w$ word vector of w when used as a **center** word. These vectors form the matrix $V \in \mathbb{R}^{d \times V}$
 - $\mathbf{u}_w$ word vector of w when used as a **context** word. These vectors form the matrix $U \in \mathbb{R}^{d \times V}$
 - These matrices are considered as model parameters θ .
- The vectors transformed to context word probabilities using **softmax**

$$P(w_o|w_c; \theta) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w=1}^V \exp(\mathbf{u}_w^\top \mathbf{v}_c)}$$

- The dot-product $\mathbf{u}_o^\top \mathbf{v}_c$ measures the similarity (alignment) of the two words.
- The exponentials makes sure the probabilities are positives, and order are preserved.
- The denominator normalizes the exponentials to a probability distributions over the **entire vocabulary** (verify that the probabilities sum to 1).

Word2Vec: optimization

- Learn (or optimize) word vectors θ so that $P(w_o|w_c)$ is maximized for positive pair (w_c, w_o) .

- (Almost) Equivalent to maximize inner product.

- Example:

- “computer vision handles images”;

- “Most languages have words as basic elements”

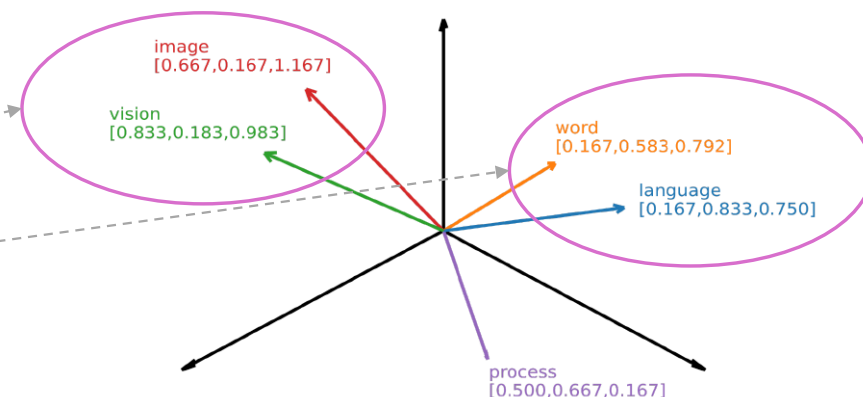
- Objective function: negative log-likelihood

$$\mathcal{L}(\theta; (w_c, w_o)) = -\log P(w_o|w_c; \theta) = -\log \exp(\mathbf{u}_o^\top \mathbf{v}_c) + \log \sum_{w=1}^V \exp(-\mathbf{u}_w^\top \mathbf{v}_c) \quad \text{Word Embeddings}$$

- Calculate the gradient and take a gradient descent step

$$\frac{\partial \mathcal{L}}{\partial \mathbf{v}_c} = -\mathbf{u}_o + \sum_{w=1}^V [\sigma(\mathbf{u}_w^\top \mathbf{v}_c)] \mathbf{u}_w$$

$$\mathbf{v}_c^{(new)} = \mathbf{v}_c^{(old)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{v}_c}$$



Word2Vec: derivation of the gradient

$$\begin{aligned}
 \mathcal{L} &= -\log \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w=1}^V \exp(\mathbf{u}_w^\top \mathbf{v}_c)} \\
 &= -\mathbf{u}_o^\top \mathbf{v}_c + \log \sum_{w=1}^V \exp(\mathbf{u}_w^\top \mathbf{v}_c)
 \end{aligned}
 \xrightarrow{1} \frac{\partial \mathcal{L}}{\partial \mathbf{v}_c} = \frac{\partial}{\partial \mathbf{v}_c} [-\mathbf{u}_o^\top \mathbf{v}_c] + \frac{\partial}{\partial \mathbf{v}_c} \left[\log \sum_{w=1}^V \exp(\mathbf{u}_w^\top \mathbf{v}_c) \right]$$

$\frac{\partial}{\partial \mathbf{v}_c} [-\mathbf{u}_o^\top \mathbf{v}_c] = -\mathbf{u}_o$

$\frac{\partial}{\partial \mathbf{v}_c} \log \sum_{w=1}^V \exp(\mathbf{u}_w^\top \mathbf{v}_c) = \frac{1}{\sum_{w=1}^V \exp(\mathbf{u}_w^\top \mathbf{v}_c)} \cdot \frac{\partial}{\partial \mathbf{v}_c} \left[\sum_{x=1}^V \exp(\mathbf{u}_x^\top \mathbf{v}_c) \right]$

$\frac{\partial \mathcal{L}}{\partial \mathbf{v}_c} = -\mathbf{u}_o + \sum_{x=1}^V \left[\frac{\exp(\mathbf{u}_x^\top \mathbf{v}_c)}{\sum_{w=1}^V \exp(\mathbf{u}_w^\top \mathbf{v}_c)} \right] \mathbf{u}_x$

$\frac{\partial \mathcal{L}}{\partial \mathbf{v}_c} = -\mathbf{u}_o + \sum_{w=1}^V [\sigma(\mathbf{u}_w^\top \mathbf{v}_c)] \mathbf{u}_w$

Word2Vec: interpretation of gradient descent

$$\frac{\partial \mathcal{L}}{\partial \mathbf{v}_c} = -\mathbf{u}_o + \sum_{w=1}^V [\sigma(\mathbf{u}_w^\top \mathbf{v}_c)] \mathbf{u}_w \quad \mathbf{v}_c^{(new)} = \mathbf{v}_c^{(old)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{v}_c}$$

- **The “Ground Truth”** $-\mathbf{u}_o$: We want to move the center vector $\mathbf{v}_c$ in the direction of the word w_o that accompnies w_c .
- **The “Expectation”** $\sum_{w=1}^V P(w_w|w_c) \mathbf{u}_w$: We subtract the average of all word vectors in the vocabulary, weighted by how much the model currently “thinks” they belong there.
 - Each word should have its own unique position and not mixed with the center of similar words (as they are easily confused with each other). Irrelevant words are not considered.

If the model predicts the correct word with 100% probability, the summation becomes $\mathbf{u}_o$, the terms cancel out, and the gradient becomes zero—meaning the model has nothing left to learn from that pair.

Word2Vec: optimization efficiency

- Negative sampling

- There are too many items in the calculation of $\sum_{w=1}^V P(w_w|w_c)\mathbf{u}_w$
- Just sample K negative pairs. $\frac{\partial \mathcal{L}}{\partial \mathbf{v}_c} = -\mathbf{u}_o + \sum_{k=1}^K [\sigma(\mathbf{u}_{w_k}^\top \mathbf{v}_c)] \mathbf{u}_{w_k}$

Like humans interact with the physical world to learn semantics, a word interacts with many other words to learn its semantics.

- Mini-batch Stochastic Gradient Descent (SGD)

- A word will appear in different windows to learn its semantics (**too many windows**)
- Mini-batch SGD: each gradient descent step only samples a small number B of positive pairs (called a "mini-batch").

$$\mathcal{L}_i(\theta) = -\log P(w_{o,i}|w_{c,i}) = -\log \frac{\exp(\mathbf{u}_{o,i}^\top \mathbf{v}_{c,i})}{\sum_{w=1}^V \exp(\mathbf{u}_w^\top \mathbf{v}_{c,i})}$$

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{i=1}^B \mathcal{L}_i(\theta) \quad (\text{mini-batch of size } B)$$

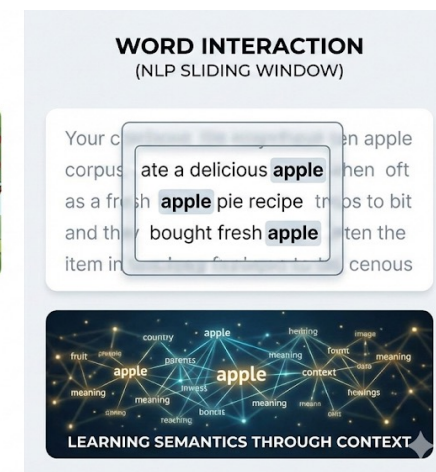
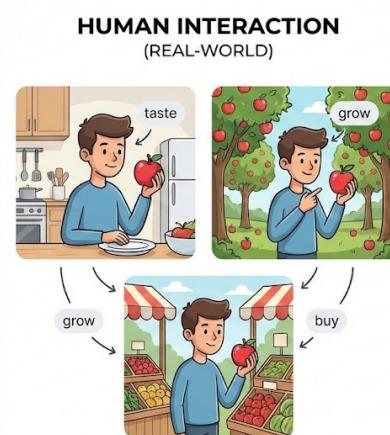


Image generated by Gemini Banana Nano

Word2Vec variant Glove

- Word2Vec learns by looking at local context windows
 - it doesn't explicitly utilize global statistics of the corpus.
 - For example, it doesn't directly "know" that "ice" appears with "steam" very rarely across the entire 10-billion-word corpus.
- Intuitions: The ratios of co-occurrence probabilities matters
 - The contrast of two probabilities remove the less relevant information.
 - What is important is the relative co-occurrence frequency, not the probability itself.
 - It is not sensitive to the scale of the probabilities.

Probability and Ratio	$k = solid$	$k = gas$	$k = water$	$k = fashion$
$P(k ice)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k steam)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k ice)/P(k steam)$	8.9	8.5×10^{-2}	1.36	0.96

image courtesy of Stanford NLP group

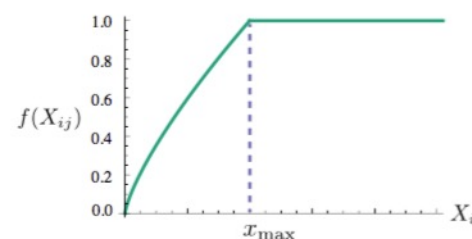
Word2Vec variant Glove

- Let the probability of word k appears in the context of word w be $P(k|w)$
- Assume that $\mathbf{w}_i^\top \mathbf{w}_k - \mathbf{w}_j^\top \mathbf{w}_k = \log \left(\frac{P(k|i)}{P(k|j)} \right)$
 - The difference in vectors model the semantic difference
 - The inner product of vectors models the predictive power (like Skip-gram)
 - The ratio of probabilities follows from the intuition from the previous slide.
 - The log leads to the exponential family distributions (that is, prob is proportional to some exponential, e.g., Gaussian distribution) – making modeling and optimization easiers.
- Objective function
 - The log of co-occurrence frequency of word i and word j is a linear function

$$\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j \approx \log(X_{ij})$$

- Make a weighted linear regression function

$$\mathcal{L}(\theta) = \sum_{i,j=1}^V f(X_{ij}) \left(\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log(X_{ij}) \right)^2$$



f : cap the frequency so that super-frequent words like stop words “the” will not dominate the loss function.

Word2Vec variant FastText

- Both Word2Vec and GloVe treat every word as an atomic entity.
 - “apple” is ID 452 and “apples” is ID 987, and they share absolutely no parameters. This leads to two major problems:
 - **Morphology is ignored:** The semantic closeness between “run”, “running” is missing.
 - **Out-of-Vocabulary (OOV) words:** newly created word (“runrunrun”) cannot be understood.
- A word is represented as a bag of its character n-grams of 3 to 6 long.
 - **Example:** “apple” becomes <ap, app, ppl, ple, le>, and the special sequence <apple>.
- Formally, let the sub-words of a center word w_c be $\mathcal{G}_{w_c}$
- The inner product between the center and context words become

$$s(w_c, w_o) = \mathbf{u}_o^\top \left(\sum_{g \in \mathcal{G}_{w_c}} \mathbf{z}_g \right) = \sum_{g \in \mathcal{G}_{w_c}} \mathbf{u}_o^\top \mathbf{z}_g$$

- The rest is similar to skip-gram

| Extra readings

- For tokenization and n-grams, see Chap 2 of SLP3.
- The Chinese room thought experiment is a good article to understand the philosophy behind language.
- For mini-batchSGD of Chap 5.9 of Deep Learning by Ian Goodfellow, etc.
- For Word2Vec, see the following articles
 - Distributed Representations of Words and Phrases and their Compositionality. Tomas Mikolov, etc. 2013.
 - GloVe: Global Vectors for Word Representation. J Pennington, etc. 2014.
 - <https://github.com/facebookresearch/fastText>

| Demon

- First generate a corpus of limited length.
- Construct vocabulary.
- Train a skipgram model based on this toy corpus.
- Visualize the word embeddings using PCA and 2D plots, to see how related words are converging together.

Conclusion

- Tokenization
- Symbolic vs. Semantics
- n-gram language models
- Word2Vec: skipgram, Glove, fasttext
 - Word embedding is later used at the input layer of LLM.
 - FastText won the tokenization war: Modern LLMs like GPT-4, Claude, and Llama have entirely abandoned whole-word tokenization and all use subword algorithms like Byte-Pair Encoding (BPE)
- Next: POS tagging; Hidden Markov Models.
- Let us know your thoughts using the QR code on Canvas

Quiz

1. Pen and paper; no compute or cellphone allowed.
 2. Turn in your answer sheet when you leave the classroom.
- **Q1 (Multiple Choice)** What is the primary computational bottleneck that Negative Sampling addresses in the training of the Skip-gram model?
 - a) The dot product operation between word vectors is too slow.
 - b) Calculating the summation over the entire vocabulary in the softmax denominator is too expensive.
 - c) Generating sliding windows from massive text corpora takes too much memory.
 - **A1: b**
 - **Q2 (True/False)** Unlike Word2Vec, which learns incrementally from local context windows, GloVe first computes a global co-occurrence matrix for the entire corpus before starting its training process.
 - **A2: True**
 - **Q3 (Multiple Choice)** How does FastText create a vector representation for "apple"?
 - a) It learns a single, atomic vector for the unique ID assigned to "apple", just like Word2Vec.
 - b) It averages the vectors of all other words that appear in the same sentences as "apple".
 - c) It sums the learnable vectors of its constituent character n-grams (e.g., <ap, app, ppl, ple, le>) and the word itself.
 - **A3: c**